

Chapter 2: Number & Character Patterns

SLIDE 1 — INCREASING NUMBER PATTERNS

What Is an Increasing Number Pattern?

An increasing number pattern prints numbers in a sequence where the number of elements increases from one row to the next.

Example

```
1
12
123
1234
12345
```

How Does It Work?

- Row 1 → Print 1 number
- Row 2 → Print 2 numbers
- Row 3 → Print 3 numbers
- Row 4 → Print 4 numbers
- Row 5 → Print 5 numbers

The number of printed values is equal to the **row number**.

C Example

```
#include <stdio.h>
```

```
int main() {
    int i, j;

    for (i = 1; i <= 5; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d", j);
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
1
12
123
1234
12345
```

Remember

Outer loop → Rows

Inner loop → Numbers in each row

SLIDE 2 — REPEATED NUMBER PATTERNS

What Is a Repeated Number Pattern?

A repeated number pattern prints the **same number multiple times within a row**.

Example

```
1
22
333
4444
55555
```

How Does It Work?

- Row 1 → Print 1 one time
- Row 2 → Print 2 two times
- Row 3 → Print 3 three times
- Row 4 → Print 4 four times
- Row 5 → Print 5 five times

Here, the **row number decides what number to print**.

C Example

```
#include <stdio.h>
```

```
int main() {
```

```

int i, j;

for (i = 1; i <= 5; i++) {
    for (j = 1; j <= i; j++) {
        printf("%d", i);
    }
    printf("\n");
}

return 0;
}

```

Output

```

1
22
333
4444
55555

```

Important Difference

In the increasing pattern:

```
printf("%d", j);
```

The **column number** is printed.

In the repeated pattern:

```
printf("%d", i);
```

The **row number** is printed.

Remember

i → Which row?

j → Which position in the row?



SLIDE 3 — CONTINUOUS NUMBER PATTERNS

What Is a Continuous Number Pattern?

A continuous number pattern keeps increasing the number across rows instead of starting again from 1 in every row.

Example

1
23
456
78910

How Does It Work?

The numbers continue from where the previous row ended.

Row 1 → 1
Row 2 → 2 3
Row 3 → 4 5 6
Row 4 → 7 8 9 10

Why Do We Need Another Variable?

The row variable `i` tells us **which row we are on**.

The inner variable `j` tells us **which position we are printing**.

Neither one alone gives us the complete continuous sequence.

So we use another variable such as `num`.

C Example

```
#include <stdio.h>
```

```
int main() {  
    int i, j, num = 1;  
  
    for (i = 1; i <= 4; i++) {  
        for (j = 1; j <= i; j++) {  
            printf("%d", num);  
            num++;  
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```

Output

1
23
456
78910

Remember

When the value must **continue across rows**, maintain a separate variable.

num → Number being printed

SLIDE 4 — CHARACTER PATTERNS

What Is a Character Pattern?

A character pattern uses letters or other characters instead of numbers.

Example

A
AB
ABC
ABCD
ABCDE

How Does It Work?

- Row 1 → A
- Row 2 → AB
- Row 3 → ABC
- Row 4 → ABCD
- Row 5 → ABCDE

The number of characters increases with the row number.

Character Arithmetic in C

Characters have numeric values internally.

We can use:

'A' + 0
'A' + 1
'A' + 2
'A' + 3

This produces:

A
B
C
D

So we can generate letters using:

```
printf("%c", 'A' + j - 1);
```

C Example

```
#include <stdio.h>
```

```
int main() {  
    int i, j;  
  
    for (i = 1; i <= 5; i++) {  
        for (j = 1; j <= i; j++) {  
            printf("%c", 'A' + j - 1);  
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```

Output

A
AB
ABC
ABCD
ABCDE

Remember

`%c` → Prints a character.

`'A' + j - 1` → Generates the next letters.

SLIDE 5 — USING LOOP VARIABLES IN PATTERNS

Which Variable Should We Print?

In pattern programs, the most important question is:

Should we print **i**, **j**, or another variable?

Print **i** When...

The value depends on the **row**.

```
printf("%d", i);
```

Example:

```
1
22
333
4444
```

Print **j** When...

The value depends on the **position inside the row**.

```
printf("%d", j);
```

Example:

```
1
12
123
1234
```

Use Another Variable When...

The value needs to continue independently across rows.

```
int num = 1;
```

Then:

```
printf("%d", num);
```

```
num++;
```

Example:

```
1
23
456
78910
```

Character Example

For letters:

```
printf("%c", 'A' + j - 1);
```

This generates:

```
A
B
C
D
E
```

Quick Comparison

Pattern	What to Print
<pre>1 12 123</pre>	<pre>j</pre>
<pre>1 22 333</pre>	<pre>i</pre>
<pre>1 23 456</pre>	<pre>num</pre>
<pre>A AB ABC</pre>	<pre>'A' + j - 1</pre>

Remember

i → Row

j → Position

num → Continuous value

Character arithmetic → Generate letters

One-Line Recap

In pattern programming, the output value depends on what changes: the row, the position, or an independent sequence.

QUESTIONNAIRES:

1. Theory MCQ

 From Slide 1 — Increasing Number Patterns

In an increasing number pattern, what does the **outer loop** usually control?

- A) The numbers being printed
- B) The rows
- C) The characters
- D) The remainder

Answer: B) The rows 

 **Hint:**

The outer loop decides **how many rows** the pattern will have.

Glossary — **outer loop**

The loop that controls the larger structure, such as the number of rows in a pattern.

2. Code Output MCQ

 From Slide 2 — Repeated Number Patterns

What will be the output?

```
int i, j;
```

```
for (i = 1; i <= 3; i++) {  
    for (j = 1; j <= i; j++) {  
        printf("%d", i);  
    }  
}
```

```
    printf("\n");  
}
```

A)

```
1  
12  
123
```

B)

```
1  
22  
333
```

C)

```
123  
123  
123
```

D)

```
1  
2  
3
```

Answer: B)

```
1  
22  
333
```



Hint:

The code prints `i`, so the **row number** is repeated.

3. Fill in the Output

 **From Slide 3 — Continuous Number Patterns**

Fill in the blank with the correct output:

```
int i, j, num = 1;

for (i = 1; i <= 3; i++) {
    for (j = 1; j <= i; j++) {
        printf("%d", num);
        num++;
    }
    printf("\n");
}
```

Output:

```
1
—
——
```

Answer:

```
1
23
456
```



 **Hint:**

The variable `num` does not restart. It **continues increasing** across rows.

4. Fill in the Missing Code

 **From Slide 4 — Character Patterns**

Complete the code to print:

```
A
AB
ABC
int i, j;
```

```
for (i = 1; i <= 3; i++) {
    for (j = 1; j <= i; j++) {
        printf("_____", 'A' + j - 1);
    }
    printf("\n");
}
```

```
}
```

Answer: %c 

 Hint:

%c is used to print a **character**.

5. Code Output MCQ

 From Slide 5 — Using Loop Variables in Patterns

What will be the output?

```
int i, j;
```

```
for (i = 1; i <= 3; i++) {  
    for (j = 1; j <= i; j++) {  
        printf("%d", j);  
    }  
    printf("\n");  
}
```

A)

```
1  
22  
333
```

B)

```
1  
12  
123
```

C)

```
123  
123  
123
```

D)

```
1
```

23
456

Answer: B)

1
12
123



 **Hint:**

The code prints `j`. Remember: `j` represents the position inside the row.